

14 Singular Value Decomposition

14.4 Truncated Singular Value Decomposition

Splitting a data matrix into a sum of rank-one matrices and approximating it with the top p components — seeing low-rank compression through an iris photo



Linear Algebra Made Easy and Visual | Open-source code & drafts: <https://github.com/Visualize-ML>

Core Skills You Will Master

1

Truncated SVD: keep the principal components, drop noise and redundancy

3

Loading = eigenvector, score = projection coordinate

5

$\mathbf{X}$ is not orthogonal in its original space, but projecting it onto $\mathbf{V}$ gives an orthogonalized $\mathbf{Z}$

2

Decompose a data matrix into a sum of rank-one matrices

4

Low-rank reconstruction: the top p components efficiently reconstruct and compress the data

From rank-one sums to low-rank approximation to orthogonal projection

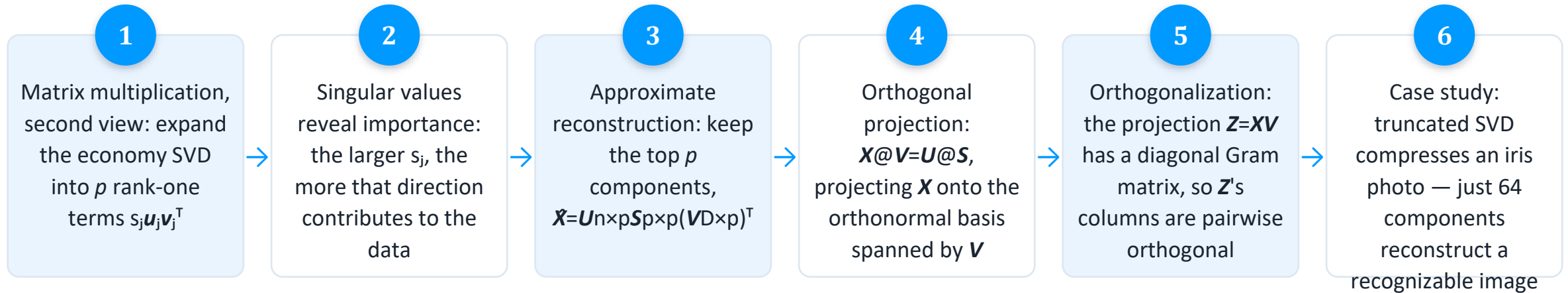


Figure 1 · Expanding the economy SVD via matrix multiplication's second view

- Expanding economy SVD $X_{n \times D} = U_{n \times D} S_{D \times D} V_{D \times D}^T$ column by column gives $X = s_1 u_1 v_1^T + s_2 u_2 v_2^T + \dots + s_D u_D v_D^T = \sum_j s_j u_j v_j^T$

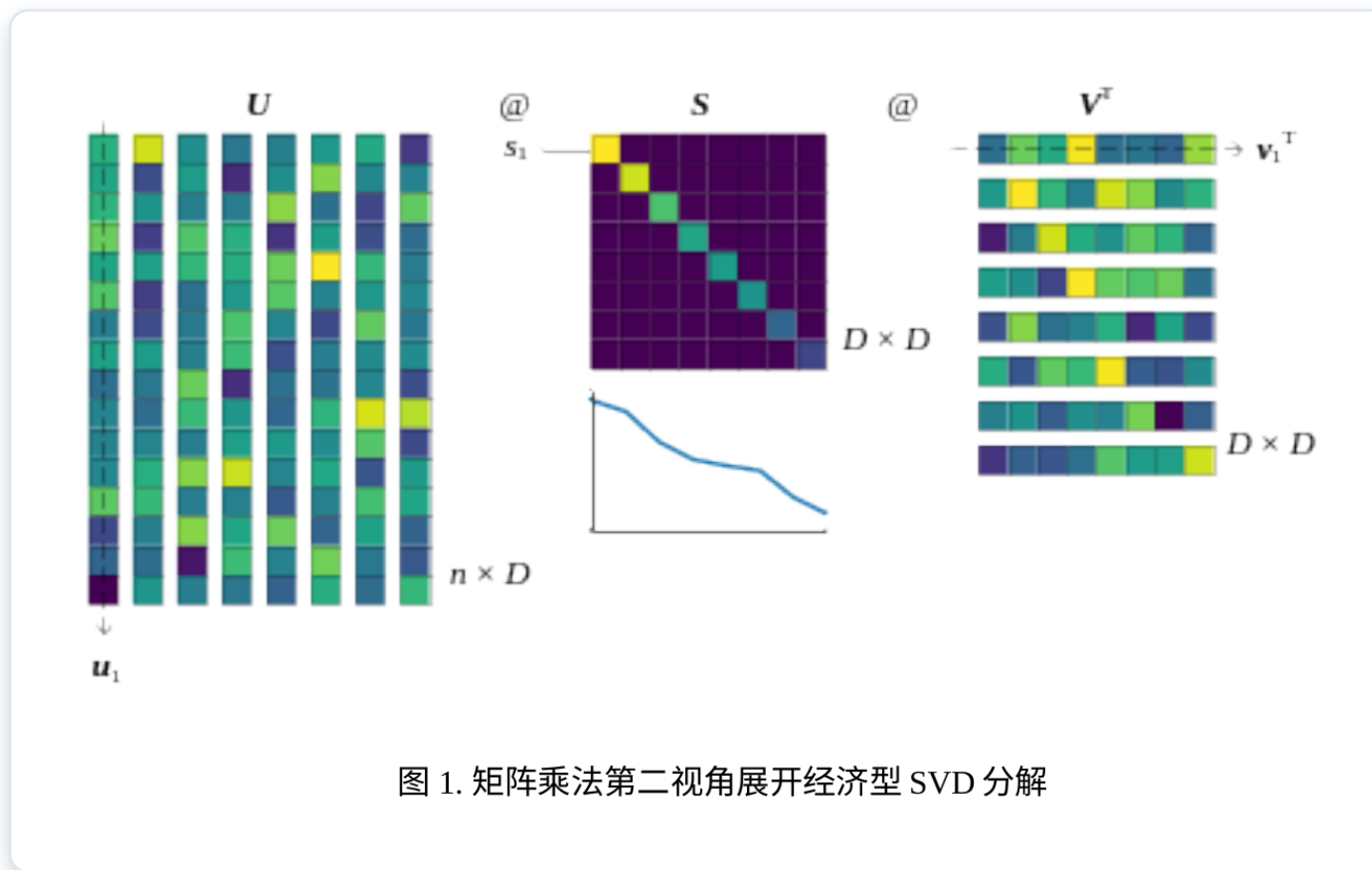


Figure 2 · The original data is the sum of D matrices of identical shape

- Each layer $s_j \mathbf{u}_j \mathbf{v}_j^T$ is a rank-one matrix the same shape as $\mathbf{X}$; stacking all D layers reconstructs the full $\mathbf{X}$

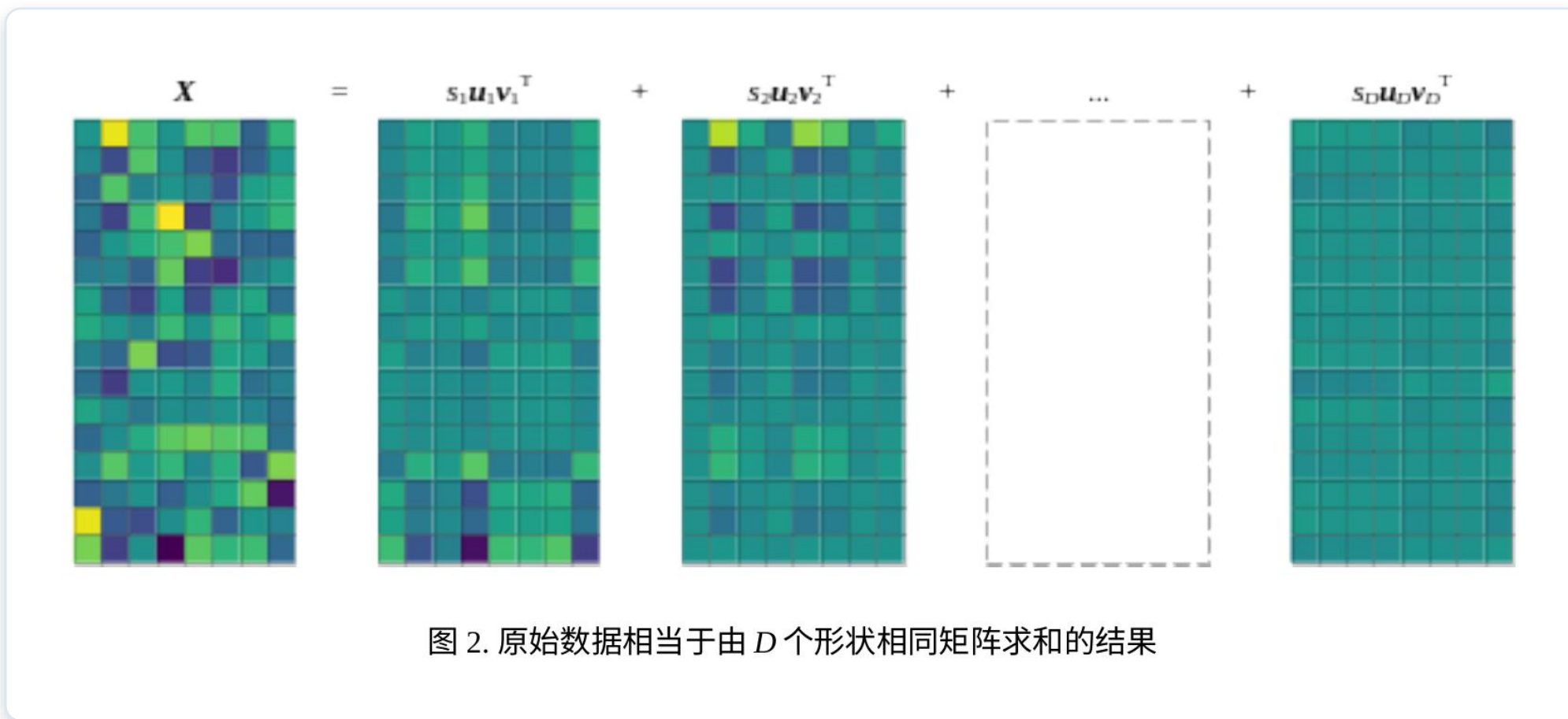


Figure 3 · Reconstructing part of the data with the top two components

- $\mathbf{u}_j$ and $\mathbf{v}_j$ are unit vectors — direction only, no size to distinguish them; the larger singular values s_1, s_2 mean these directions contribute more to the original data

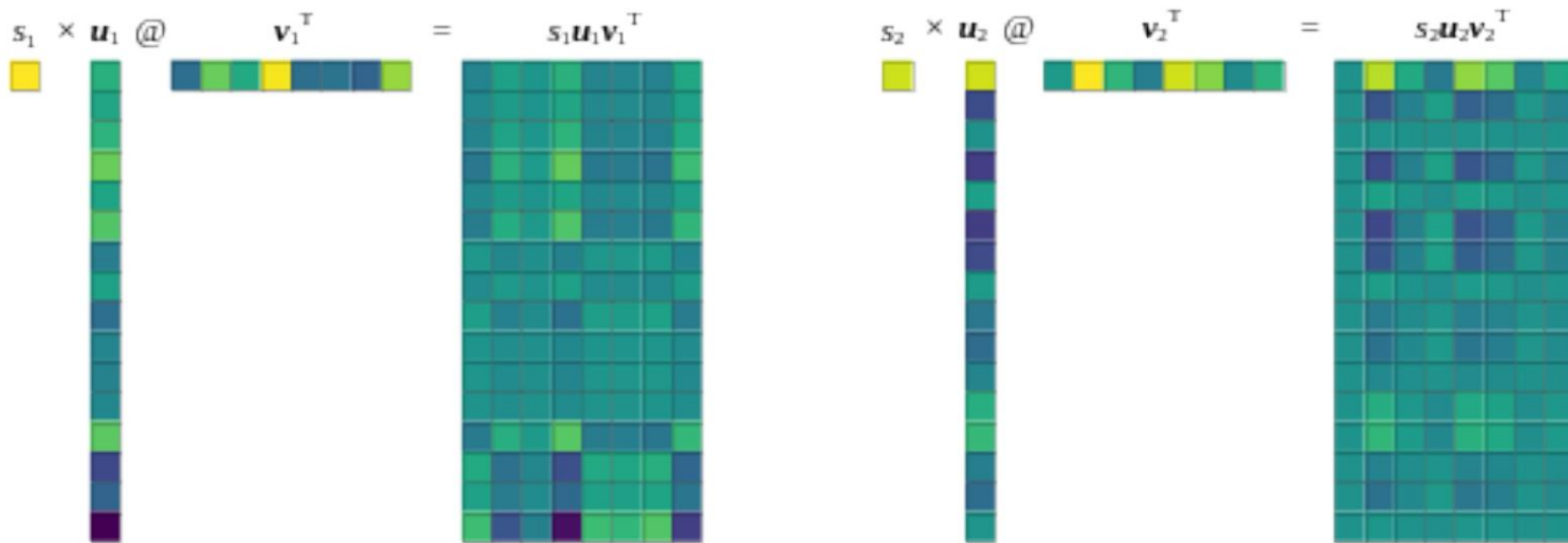
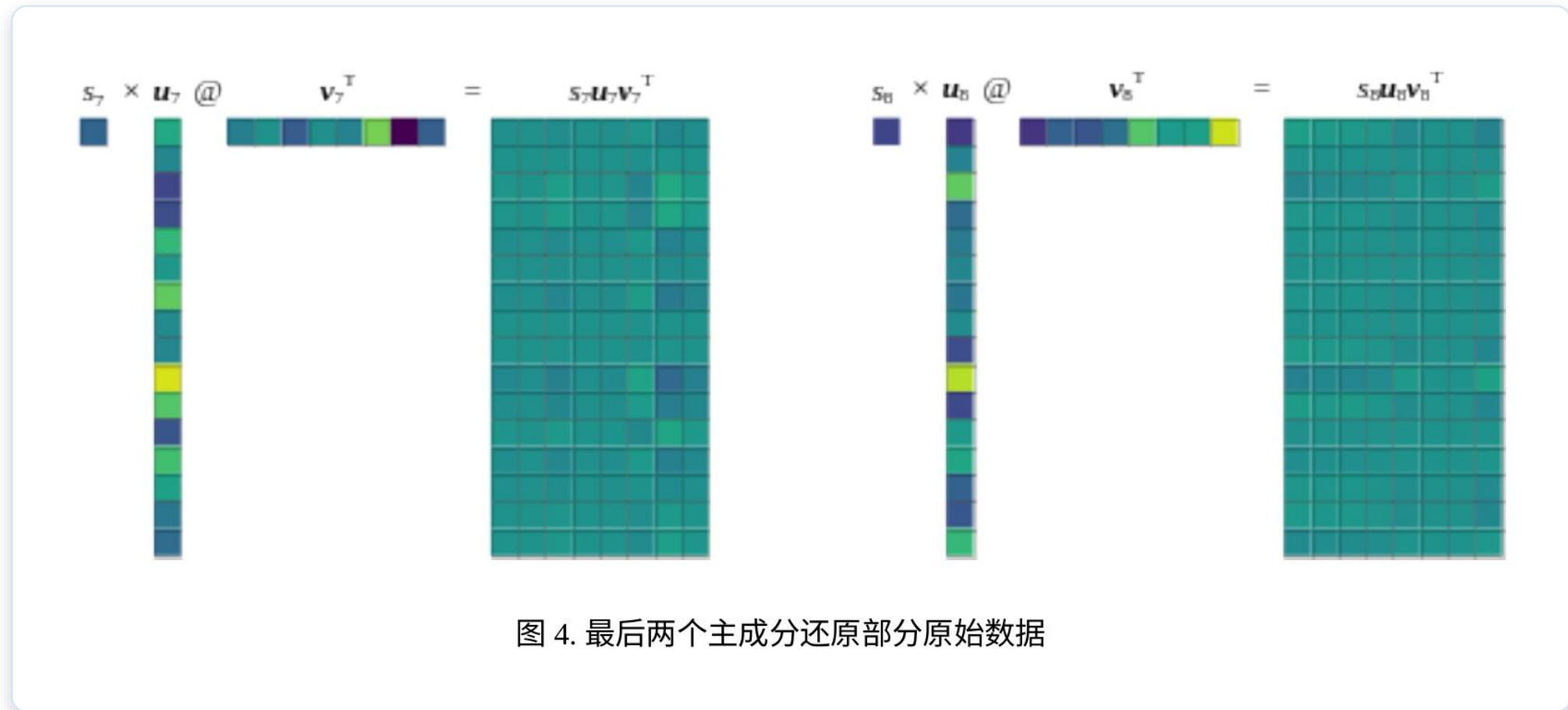


图 3. 前两个主成分还原部分原始数据

Figure 4 · Reconstructing part of the data with the last two components

- The last two layers $s_{D-1}\mathbf{u}_{D-1}\mathbf{v}_{D-1}^T$ and $s_D\mathbf{u}_D\mathbf{v}_D^T$ have tiny singular values, so their contribution is almost imperceptible



Loading = eigenvector, score = projection coordinate

- ① If singular values $s_1, s_2 \dots s_D$ are sorted from largest to smallest, $\mathbf{v}_1$ is the loading of the first principal component and $\mathbf{u}_1$ is its factor score
- ② The diagonal entries of $\mathbf{S}$ can be viewed either as scaling $\mathbf{U}$'s columns, or as scaling $\mathbf{V}^T$'s rows
- ③ Note: some references define loading as “eigenvector $\times$ singular value” — a different convention from this section, so watch for it
- ④ Different Python functions may flip the sign of the resulting unit vectors; $\mathbf{u}_j$ and $\mathbf{v}_j$ flipping together leaves $s_j \mathbf{u}_j \mathbf{v}_j^T$ unchanged

Figure 5 · Reconstructing the original data with the top p components

- $X_{n \times D} \approx \hat{X}_{n \times D} = U_{n \times p} S_{p \times p} (V_{D \times p})^T = s_1 u_1 v_1^T + \dots + s_p u_p v_p^T$; as long as the top p singular values are all positive, X has rank p

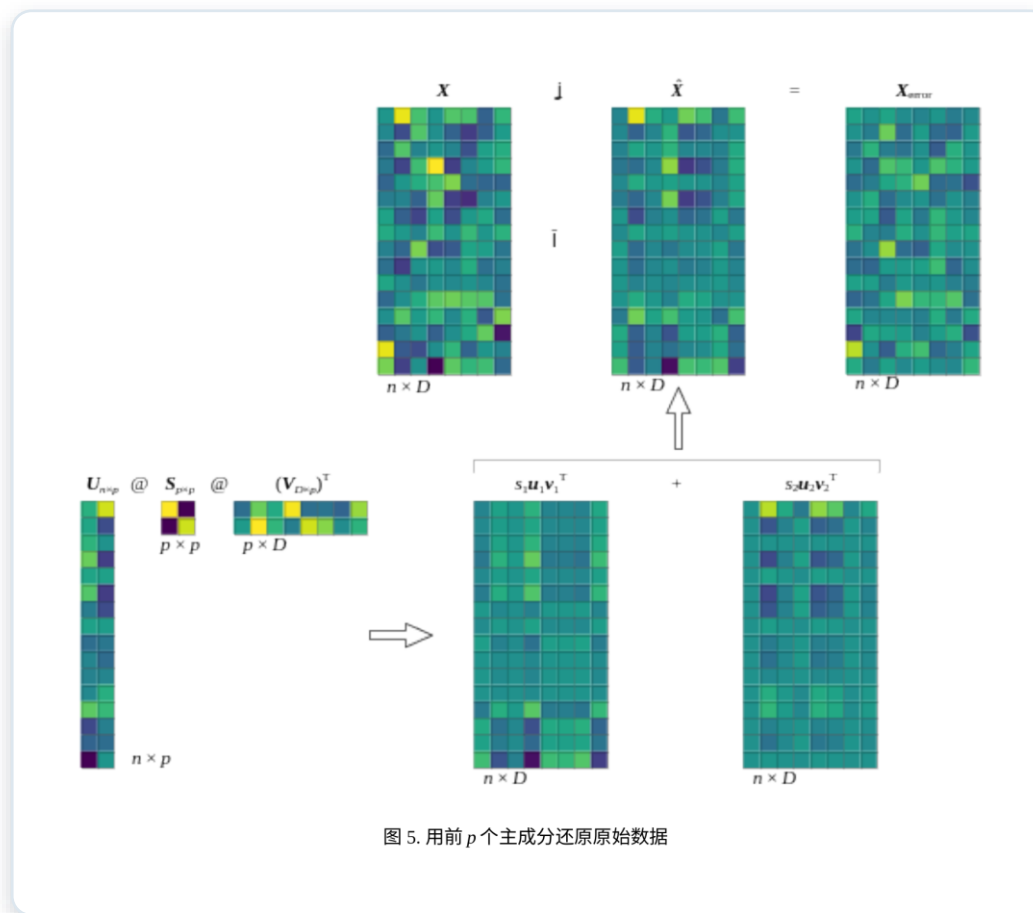


Figure 6 · Projecting the original data onto V

- $XV = US$; expanding both sides gives $Xv_j = s_j u_j$, which is exactly projecting X onto the space spanned by V

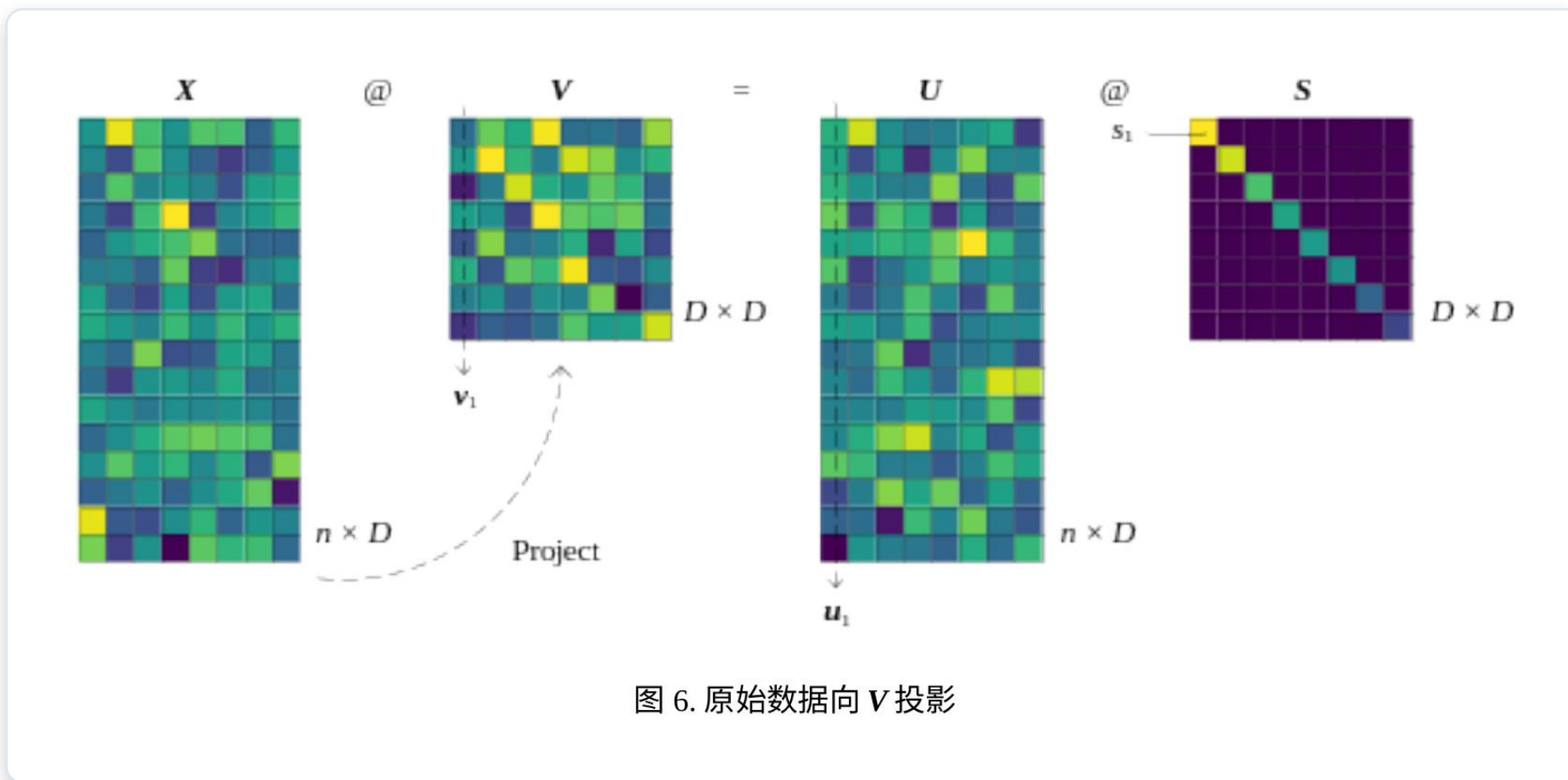


Figure 7 · Projecting the original data onto the scalar v_1

- $Xv_1 = s_1u_1$; since $\|u_1\|=1$, the projection's norm $\|Xv_1\| = s_1$ — the singular value is exactly the projected length

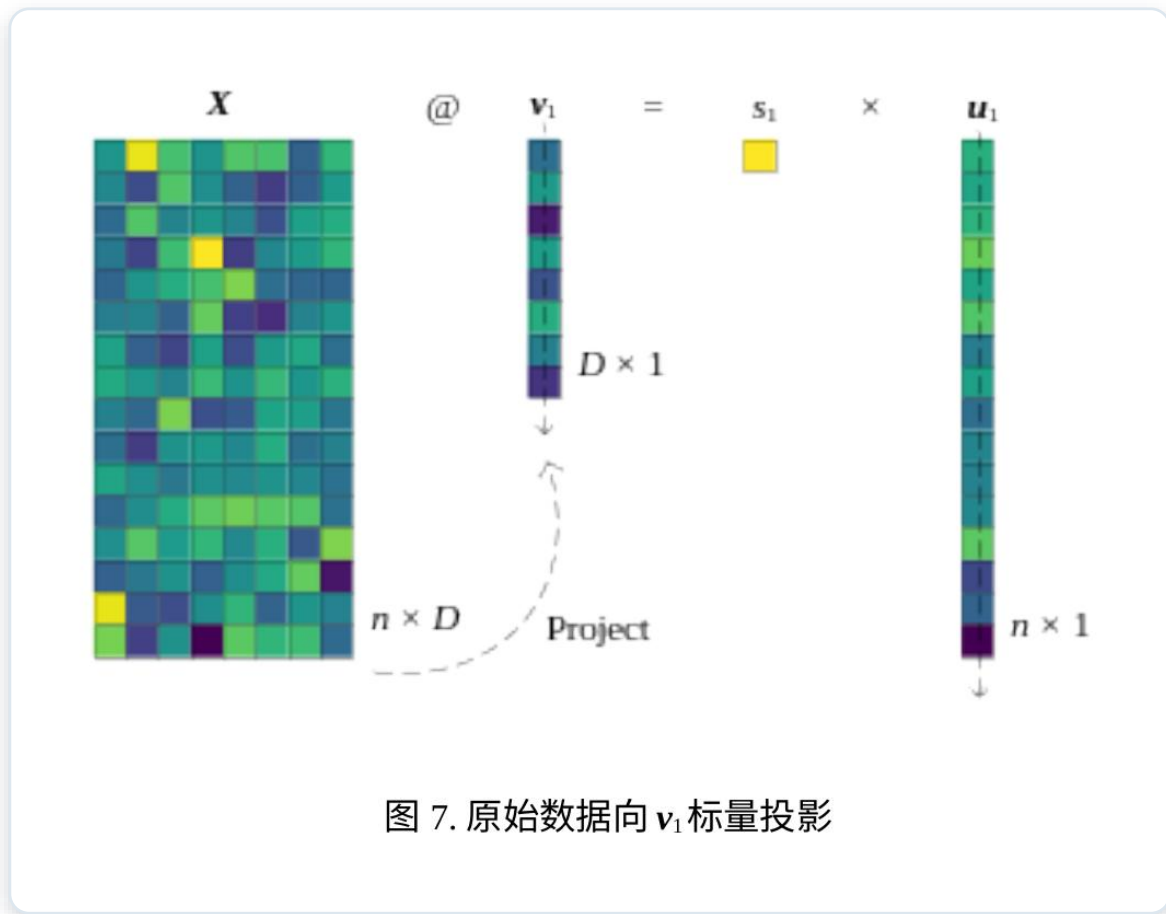


Figure 8 · Projecting X onto the orthonormal basis V

- Projecting the data matrix X onto the D -dimensional space spanned by the orthonormal basis $V=[v_1, v_2, \dots, v_D]$ gives $Z = XV$

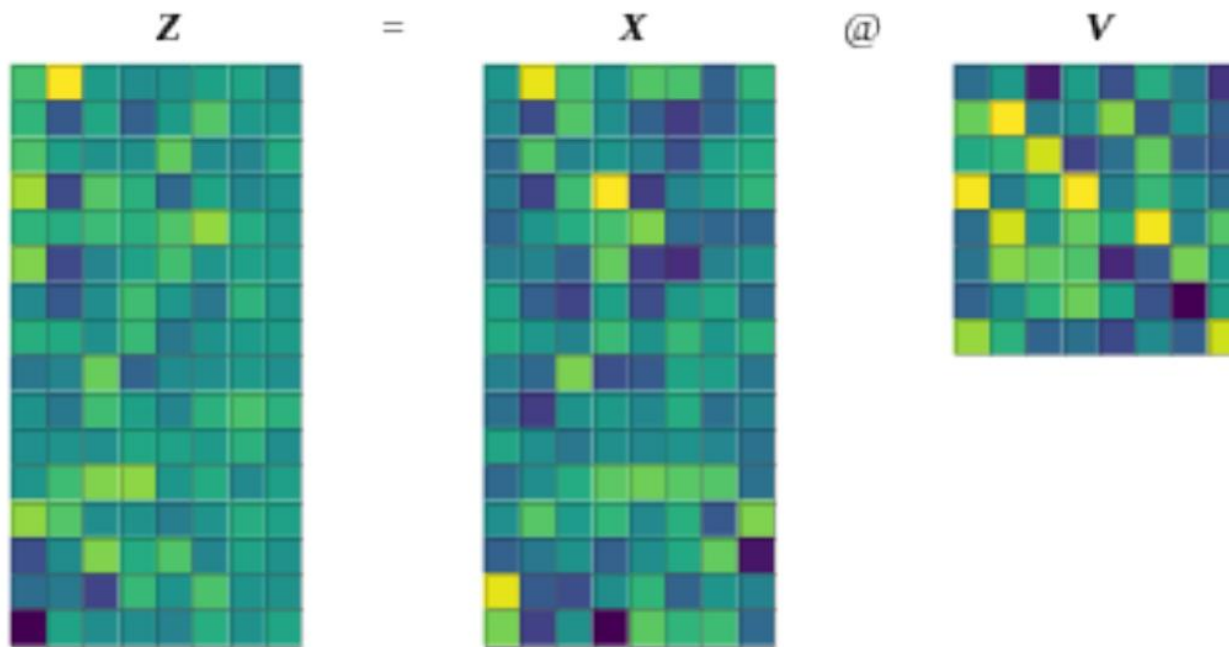


图 8. X 向规范正交基 V 投影

Figure 9 · A second path for computing Z

- Substituting $X=USV^T$ gives $Z=USV^TV=US$; u_j is the “unit-normalized” projection coordinate, and s_j is the projected vector's norm

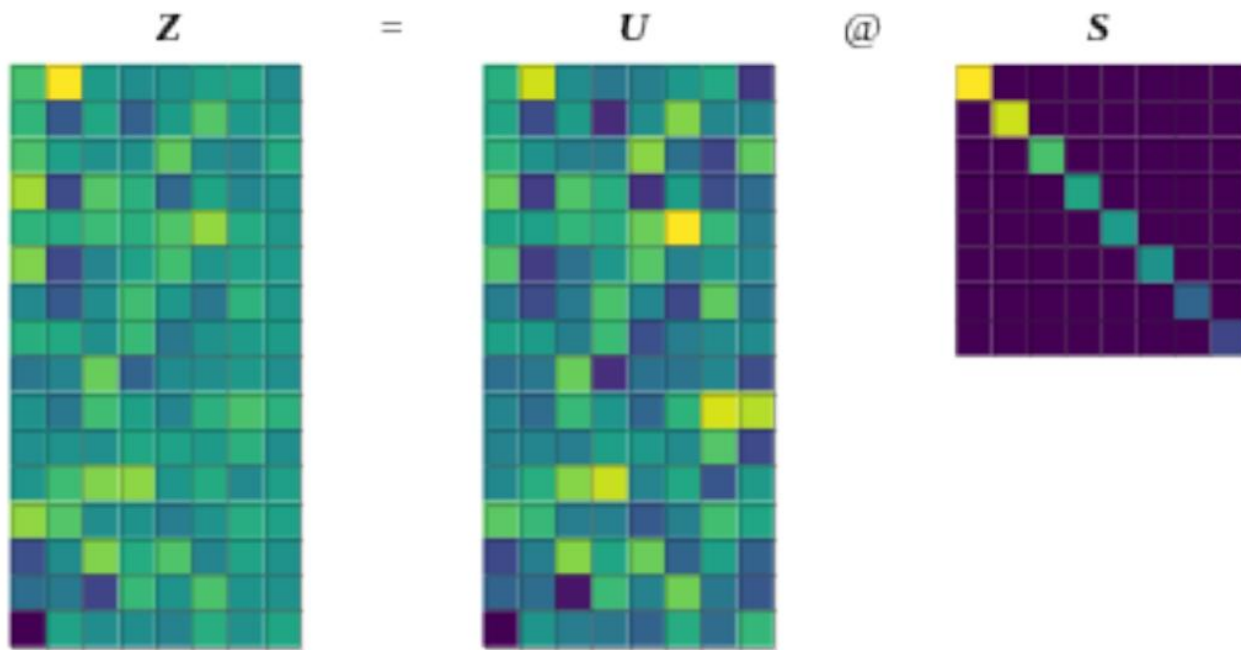


图 9. 第二条计算 Z 的路径

Figure 10 · The Gram matrix of Z

- $Z^T Z = (US)^T US = S^T S$ is diagonal, so Z 's columns are pairwise orthogonal — the map $X \rightarrow Z$ is exactly an orthogonalization

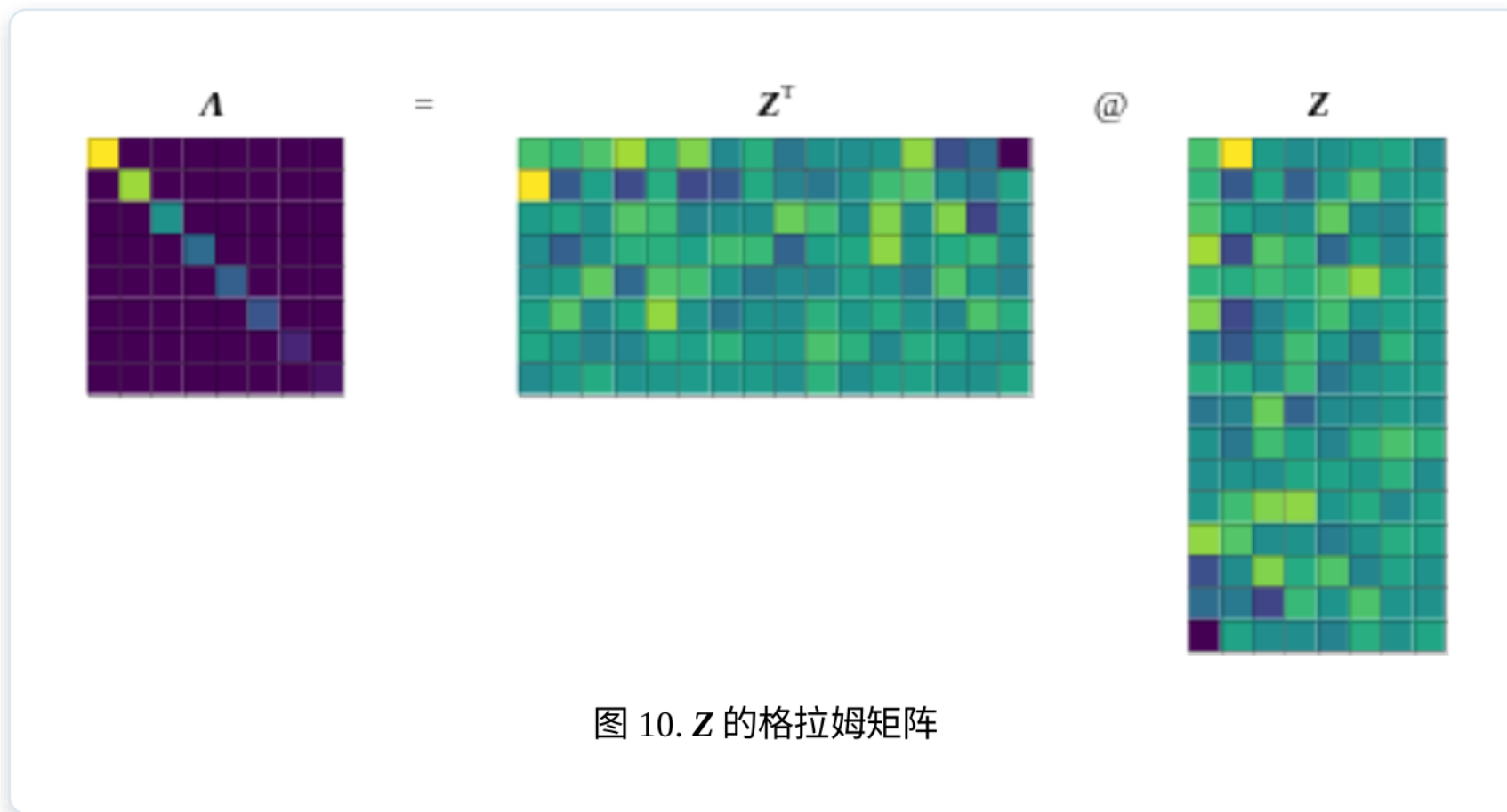


Figure 11 · An iris photo, converted to grayscale

- After converting the author's iris photo to grayscale, every pixel is a number in $[0,1]$ — the whole image can be treated as one data matrix

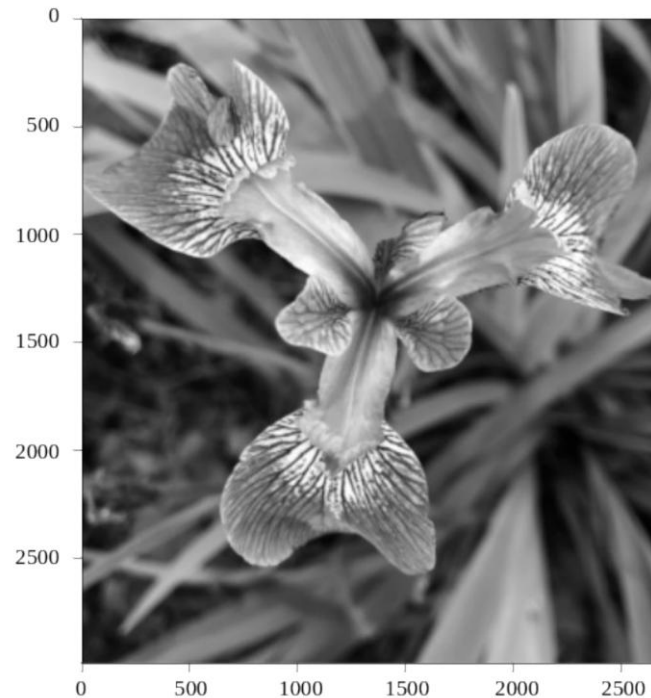


图 11. 鸢尾花图片，经过黑白处理，来自《机器学习》

Figure 12 · Singular values across principal components

- Running SVD on the image matrix, the top 10 singular values contribute the most and dominate the image's structure, then decay rapidly

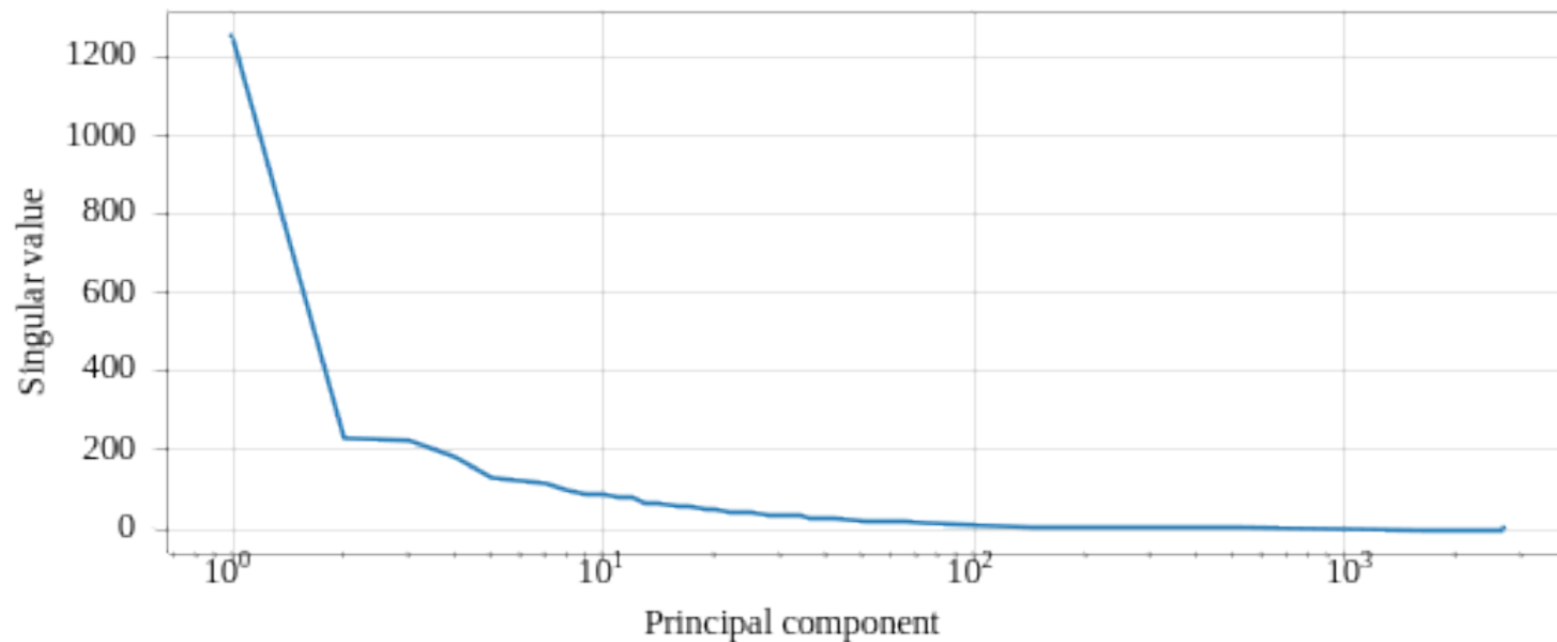


图 12. 奇异值随主成分变化，来自《机器学习》

Figure 13 · Reconstructing the iris photo with the 1st-4th components

- With only the top 4 components, the reconstruction shows just large blocky “checkers” — not yet recognizable as an iris

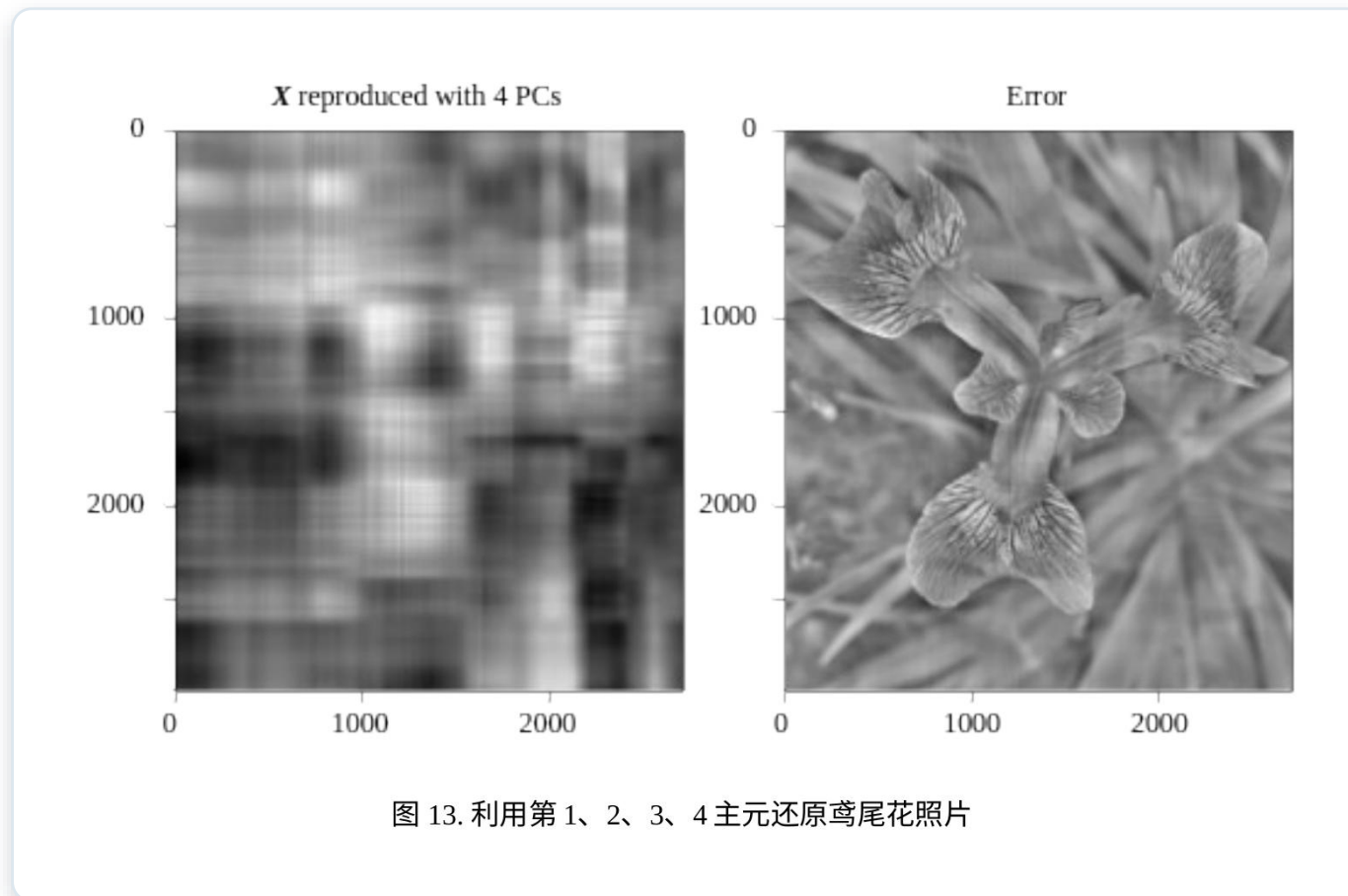


Figure 14 · Sum of the top 4 rank-one matrices

- Figure 13's reconstruction is exactly the sum of 4 rank-one matrices: $s_1\mathbf{u}_1\mathbf{v}_1^T$, $s_2\mathbf{u}_2\mathbf{v}_2^T$, $s_3\mathbf{u}_3\mathbf{v}_3^T$, $s_4\mathbf{u}_4\mathbf{v}_4^T$

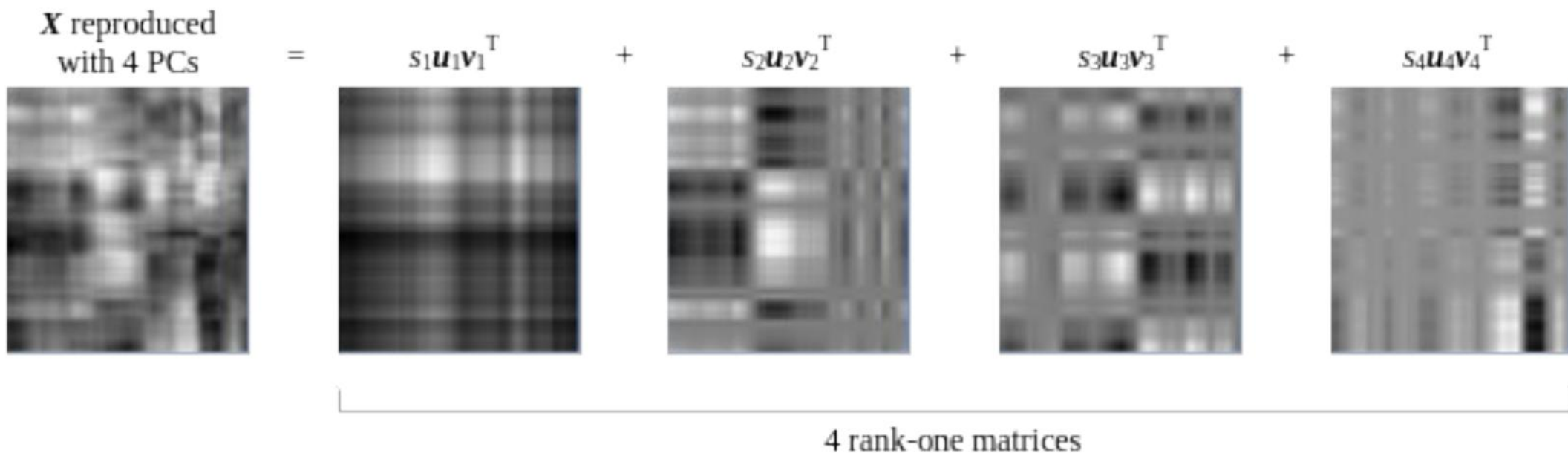


图 14. 前 4 个秩一矩阵叠加

Figure 15 · Reconstructing the iris photo with the top 64 components

- Keeping the top 64 components, the image is already clearly recognizable; compared to the original, only about 2.5% (64/2714) of the data is kept — a striking compression ratio

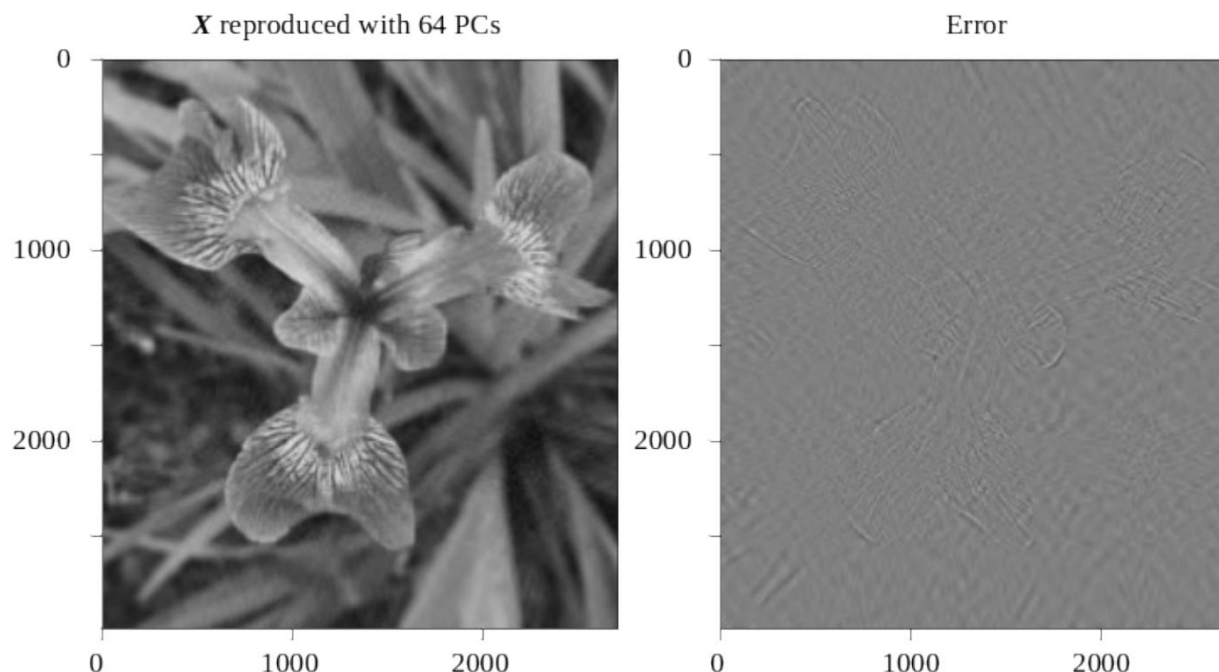


图 15. 利用前 64 个主元还原鸢尾花照片

SUMMARY

Truncated SVD: three threads — rank-one sums, approximation, and projection

① The economy SVD expands column by column into a sum of D rank-one matrices: $\mathbf{X} = \sum_j s_j \mathbf{u}_j \mathbf{v}_j^T$

② Singular value s_j reveals a direction's importance; sorted largest to smallest, $\mathbf{v}_1$ is the first loading and $\mathbf{u}_1$ is the first score

③ Truncated SVD keeps only the top p components:
 $\mathbf{X} \approx \mathbf{U}_{n \times p} \mathbf{S}_{p \times p} (\mathbf{V}_{D \times p})^T$ — an approximation of $\mathbf{X}$, not an exact reconstruction

④ The error matrix $\mathbf{X}_{\text{error}}$ is itself a sum of the dropped layers, from $p+1$ through D

⑤ Projection view: $\mathbf{Z} = \mathbf{XV} = \mathbf{US}$; $\mathbf{u}_j$ is the “unit-normalized” projection coordinate, and s_j is the projected length

⑥ $\mathbf{Z}$'s Gram matrix $\mathbf{Z}^T \mathbf{Z}$ is diagonal — the map from $\mathbf{X}$ to $\mathbf{Z}$ is exactly an orthogonalization

Key Terms in This Section

Rank-One Matrix *A matrix of the form $s_j u_j v_j^T$, always rank 1 — the basic building block of SVD*

Score *The projection coordinate u_j for the j -th principal component, i.e. the factor score*

Gram Matrix *$Z^T Z$, used to test whether Z 's columns are pairwise orthogonal*

Loading *The direction vector v_j for the j -th principal component, i.e. the eigenvector*

Truncated SVD *Keeps only the components for the top p largest singular values — an approximation ($\approx$) of the original matrix*

Orthogonalization *The process of projecting X onto V to get Z , whose columns are pairwise orthogonal*

FURTHER READING

Further Reading

Review · Chapter 3, Section 2 — Matrix Multiplication's Second View

Expanding matrix multiplication as a sum of outer products — the foundation for this section's derivations

Review · Chapter 11, Section 4 — Spectral Decomposition of the Gram Matrix

Understand the orthogonalization logic behind $Z^T Z$ being diagonal

Review · Sections 14.1 – 14.3, The Four Flavors of SVD

Full, economy, and compact SVD are the foundation for truncated SVD

Open-source Code Repository

Code and figures for the entire book: <https://github.com/Visualize-ML>

Exercises Q1 – Q9

Q1 Work out the economy SVD of $A = \begin{bmatrix} 0 & 1 \\ 1 & 1 \\ 1 & 0 \end{bmatrix}$, keep only the first principal component to approximate A , and compute the error.

Q3 For the A in Q1, write out the loading and score for v_1 and u_1 , and explain their geometric meaning.

Q5 Compute the error matrix X_{error} after keeping only the first component of the Q1 matrix A , and verify $X_{\text{error}} = s_2 u_2 v_2^T$.

Q7 Design a 4×3 matrix, compute its economy SVD in Python, and plot a bar chart of the singular values from largest to smallest.

Q9 Explain why loading times score (the outer product of v_j and u_j , scaled by s_j) must always be a rank-one matrix.

Q2 For the A in Q1, write its rank-one sum decomposition and verify in Python that the sum equals A .

Q4 In Python, verify that the two rank-one matrices $s_1 u_1 v_1^T$ and $s_2 u_2 v_2^T$ of the Q1 matrix A sum exactly to A .

Q6 In Python, verify that the L2 norm of u_1 is 1, and that $\|A v_1\| = s_1$.

Q8 For the matrix in Q7, reconstruct it with the top 1, 2, and 3 components, and compare how the reconstruction error's Frobenius norm changes.

Exercises Q10 – Q17

Q10 In Python, verify that projecting the data matrix X onto the orthonormal basis V gives a Z whose Gram matrix $Z^T Z$ is diagonal.

Q12 Explain why u_j can be called the “unit-normalized” projection coordinate, using $\|Xv_j\|=s_j\|u_j\|=s_j$ to support your answer.

Q14 Using the image from Q13, reconstruct it keeping the top 4, 16, and 64 components, and compare how visual clarity changes.

Q16 Using Matplotlib, plot the heatmap of the error matrix X_{error} for $p=4$ in Q14, and analyze the error pattern in the style of Figure 9.

Q11 For the matrix in Q7, compute $Z=XV$ and verify that Z 's columns are pairwise orthogonal.

Q13 In Python, load a grayscale image, compute its singular values, and plot how they vary across principal components, in the style of Figure 12.

Q15 Compute the compression ratio (retained data / original data) for $p=64$ in Q14, and compare it with the $64/2714$ result in Figure 15.

Q17 Explain why different Python functions may return u_j, v_j with opposite signs, yet $s_j u_j v_j^T$ always comes out the same.

Exercises Q18 – Q25

Q18 Design a 5×4 matrix and write out its economy SVD expanded as a full sum of rank-one matrices.

Q20 In Python, verify that the squared Frobenius norm of the truncated SVD's error matrix X_{error} equals the sum of squares of the dropped singular values.

Q22 Explain the alternative convention “loading = eigenvector $\times$ singular value” mentioned in this section, and compare how the two definitions affect applications.

Q24 Give an application of truncated SVD in recommender systems or natural language processing, and explain the role of low-rank approximation.

Q19 For the matrix in Q18, compute the approximations for $p=1, 2, 3$ components, and verify each has rank exactly p .

Q21 Drawing on orthogonal projection matrices from Chapter 9, explain the geometric meaning of the projection $Z=XV$ in Figure 8.

Q23 In Python, compute the rank-one decomposition of a full-rank matrix and a rank-deficient matrix, and compare how many nonzero terms each has.

Q25 Summarize how this section's three perspectives — rank-one sums, approximate reconstruction, and orthogonal projection — relate to and differ from one another.

10 Prompts to Explore with AI

- 1 Explain why the economy SVD can be expanded as a sum of D rank-one matrices.
- 2 Explain how the size of singular value s_j reflects the importance of its corresponding direction to the original data.
- 3 Explain exactly what “loading = eigenvector, score = projection coordinate” means.
- 4 Give an example showing why truncated SVD produces an approximation rather than an exact reconstruction.
- 5 Explain how the truncated SVD's error matrix X_{error} is built from the dropped singular-value components.

10 Prompts to Explore with AI

6 Explain the geometric meaning of projecting the data matrix X onto the orthonormal basis V to get Z .

7 Explain why Z 's Gram matrix is diagonal, and what that tells us.

8 Using the iris photo example, explain the practical effect of truncated SVD in image compression.

9 Explain why different Python functions may return singular vectors with different signs, yet this doesn't affect the final result.

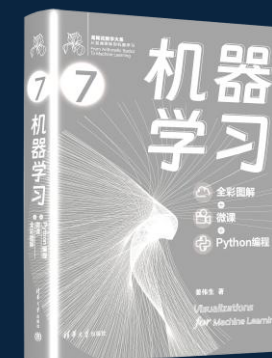
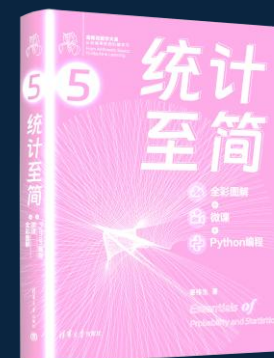
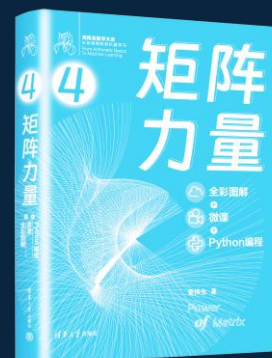
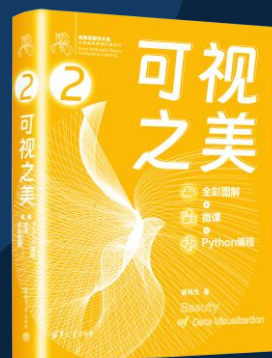
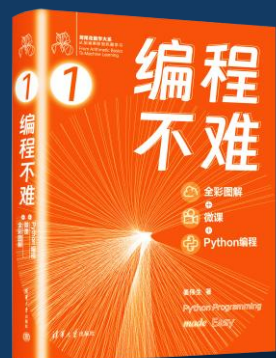
10 Compare the relationship between truncated SVD and principal component analysis (PCA).

数学

数学基础

线性代数

概率统计



Python编程

数据可视化

工具

数据分析
图和网络

回归、降维
分类、聚类

实践

